Class 5: LLM Behavior, Prompting & Retrieval

Personal Wiki with Local Gemma + RAG | From Zero to AI Agents

Class signup

Assignment setup

Start these downloads now for your personal wiki.

- **Obsidian:** install it to view your wiki, links, and graph.
- **Apple Silicon Mac (M-series):** use **MLX via `mlx-vlm`**.
- **Windows / Linux:** install **Ollama**.
- **Local Gemma:** start with **Gemma 4 E2B, 4-bit**, if it fits your memory. Download packages and any embedding model while online.

  Intel Mac: **Ollama supports CPU inference on macOS 14+**; expect slower replies.

Class signup

Scan or open the form ↗

Prompt for your coding assistant

> Check my OS, chip, RAM/VRAM, and free disk space. Set up a compatible local Gemma runtime and dependencies, download a small model, and help me install Obsidian. Verify a local reply with the internet disconnected.

Pac-Man: the top five

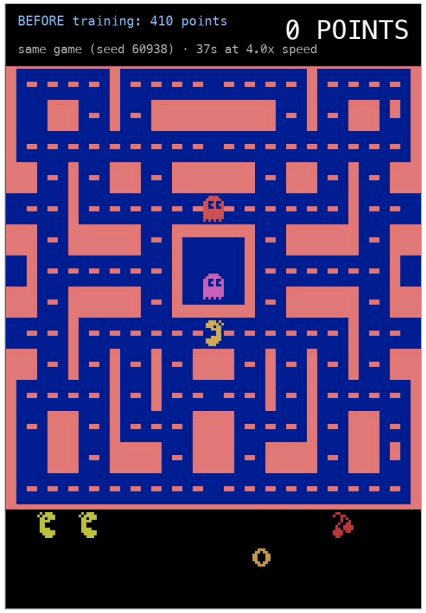
Best submitted runs

Highest single-game scores

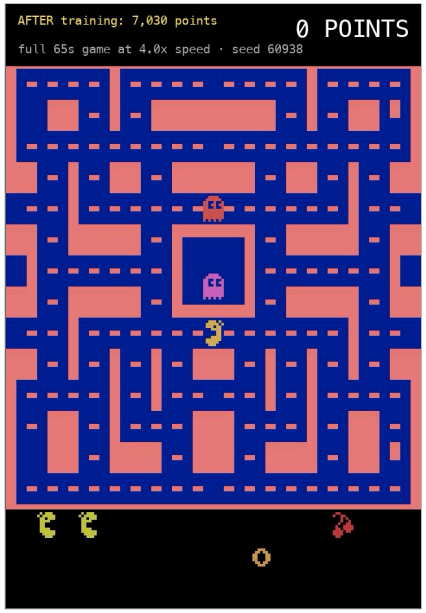
Student	Points
1. Sebastián Bardacosta	**7,030**
2. Hanif Ramadhan	3,110
3. Matthew Wong	2,380
4. Geronimo Molina	2,230
5. Amelia Roeschel	2,150

Includes updated demonstrations. Game seeds vary.

Before training



After training



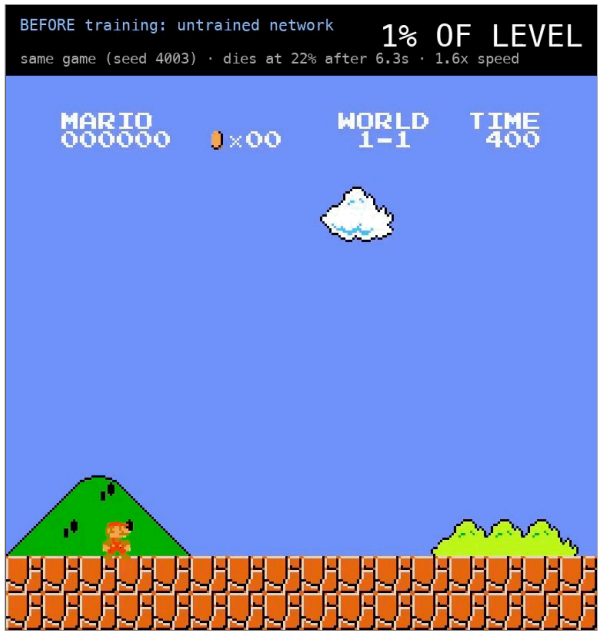
410 to 7,030 points

Same game, seed 60938. Full runs at 4× speed.

Super Mario: before and after

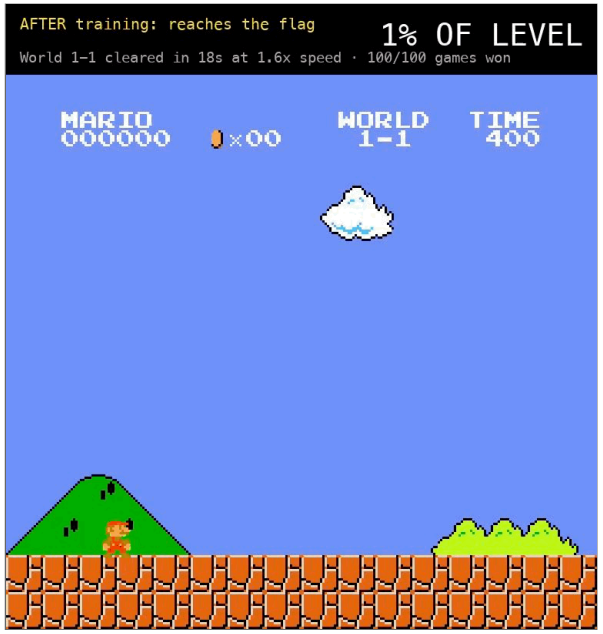
Learning to clear **World 1-1**

Before training



Reaches 22% of the level

After training



Clears World 1-1 in 18 seconds

Sebastián's Mario project

What This Class Gives You

- **Prompting** — how to steer the probability distribution, with or without thinking mode, so you get useful answers the first time
- **Context** — why the model forgets, what compaction does, why this limits every AI feature you'll ship
- **Fine-tuning, RLHF, alignment** — what labs actually do under the hood, so *"Anthropic values"* and *"OpenAI personality"* become real design choices, not magic
- **RAG** (Retrieval Augmented Generation) — how to bolt your company's data onto a general model **without** paying $100M to retrain
- **Search & tools**: how AI finds relevant passages and looks up facts in databases or on the web
- **Assignment 4: Personal Wiki CLI**: build your own harness around **local Gemma + RAG**, chosen for your computer, to answer from your data **offline**

  Building agents, connecting tools, MCP, and evals are **next class**. Today's personal wiki is a complete, standalone project.

> **Important**
>
> You won't be an ML engineer. You'll be the MBA who **ships** working AI workflows instead of the one who waits for IT.

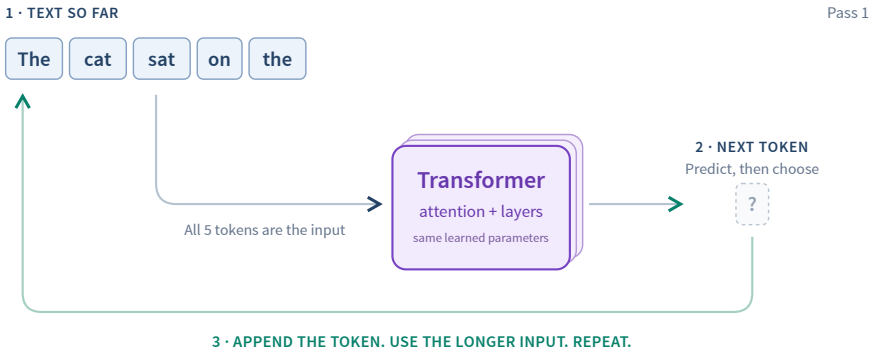
Agenda

1. **Recap** — LLMs predict tokens, probabilistically
2. **Fine-tuning & RLHF** — how a base model becomes an assistant, and what it costs
3. **Prompting** — probability steering, failure modes, voice prompting
4. **Better prompts**: speak the context, include useful detail, and make a clear ask
5. **Context windows** — LotR-sized memory, compaction, token economics
6. **Prompt engineering** — .md files, system prompts, CLAUDE.md, the golden rule
7. **Thinking models** — when the model plans before it talks
8. **Hallucination & RAG**: grounding answers in evidence, with an animated pipeline
9. **Search & tools**: chunking, vector databases, SQL, and web search
10. **Assignment 4: Personal Wiki CLI**: choose Gemma for your device, build your own RAG harness

Class 4 Recap

Slide 1 of 2: LLMs Predict Tokens

Each new token becomes part of the next input.



Start with the prompt. The model will add one token at a time.

Play Step Restart

Illustrative sequence. Tokens are shown as words and punctuation; real tokens can also be word pieces.

Slide 2 of 2: LLMs Are Probabilistic

The model does not produce one guaranteed answer.
It produces a **probability distribution** over possible next tokens, then samples from it.

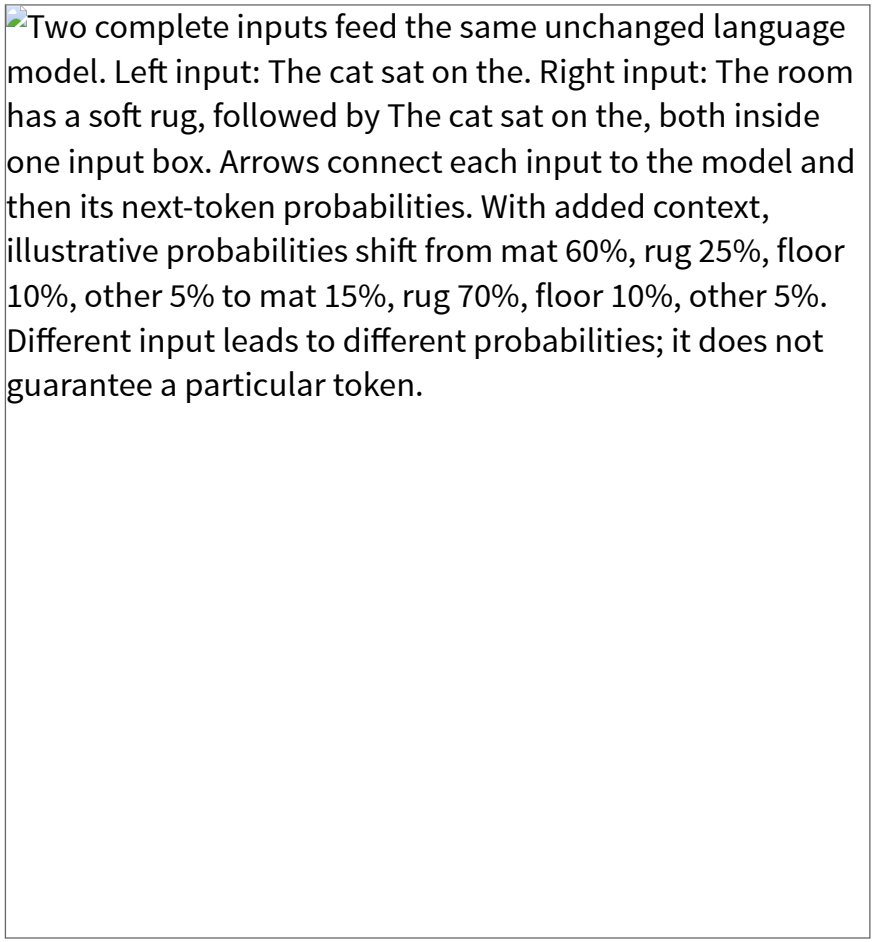
- same prompt can produce different answers
- better prompts reshape the distribution
- more context changes what becomes likely
- exact tasks still need deterministic code

“It Knows” Is a Risky Shorthand

An LLM contains learned statistical structure in its parameters.
It does not consult a verified, current, cited database unless you give it one.
That single sentence explains most of what this class covers:

- **why prompting works** — you’re reshaping which continuation is likely
- **why hallucination happens** — plausible ≠ verified
- **why RAG exists** — to bolt a real, cited knowledge source onto the statistical machine

The Probability Distribution Is the Steering Surface



Practical Implications

If Class 4 explained the machine, Class 5 is about how to use it well.

- **Why prompting matters**: prompts steer the probability distribution
- **Why exact tasks need code**: probabilities are not enough for precise pipelines
- **Why context matters**: the model only sees what you put in the window

Temperature: The Creativity Dial

Every API call includes a **temperature** setting — a number that controls how “random” the output is.

	Low temp	High temp
Good for	facts, code, extraction	brainstorming, writing, ideas
Risk	repetitive, boring	hallucinations, nonsense

> **Note**
>
> Temperature changes **variety, not intelligence**. Lower temperature favors more likely continuations; higher temperature explores more possibilities and raises variance. It doesn’t make the model smarter or dumber — it changes how adventurous the sampling is.

What Is Fine-Tuning?

Fine-Tuning Reshapes the Distribution

In Class 4 we saw you can take a pretrained model and **fine-tune** it — train it further on specific data.
But what does that actually mean? It **shifts which tokens the model considers likely**.

Same prompt, completely different predictions — because the weights changed.

> **Note**
>
> Fine-tuning is **expensive and permanent** — it changes the model's weights. Prompting is **cheap and temporary** — it steers the same frozen model. Most of this class is about prompting, because that's what you'll use day-to-day.

RLHF — How Models Learn to Be Helpful

Fine-tuning on examples makes a model follow instructions. But **how does it learn to be helpful, polite, safe?** Not from raw internet. From humans.
Post-training is three stages — and each one uses a learning paradigm from Classes 3 and 4:

1. **Pretraining** (*unsupervised*) — predict next token from raw internet. No labels — just patterns in text. Knows everything, does nothing useful.
2. **SFT — Supervised Fine-Tuning** (*supervised*) — labeled *instruction → good-response* pairs. This is what most people call "fine-tuning."
3. **RLHF — Reinforcement Learning from Human Feedback** (*reinforcement*) — remember Pac-Man from Class 3? Same loop: try an action, get a reward, learn. Only now the "reward" is **human preference** (*A vs. B, which is better?*), encoded in a **reward model** and optimized via **PPO** or **DPO**.

> **Tip**
>
> **Callback to Classes 3 and 4.** The same three learning paradigms you met earlier — unsupervised, supervised, reinforcement — are **literally the three stages** of training a modern LLM. In order.

Important

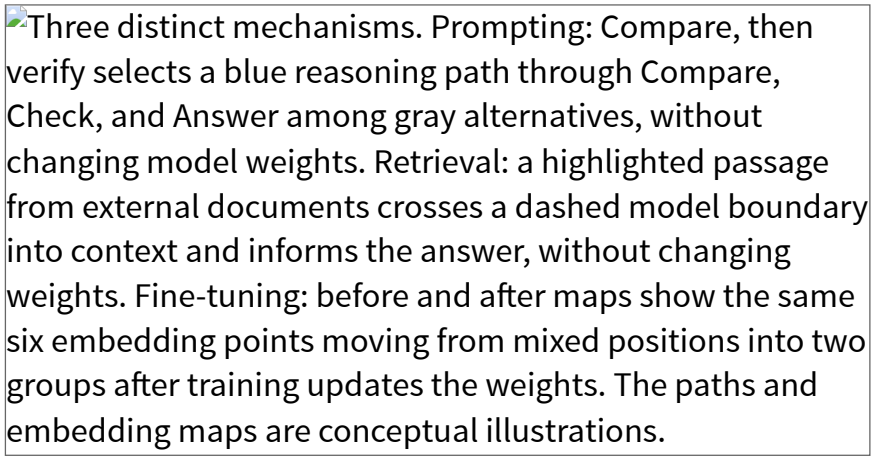
MBA lens. *A prompt can't override what RLHF taught.* That's the difference between **personality** (prompt-level, changeable per-session) and **values** (weight-level, baked in forever). Firms pick an RLHF recipe the same way they pick a hiring rubric — it determines who the model becomes.

(In Class 7 we'll see why Anthropic, OpenAI, and Character.AI all hire philosophers for this step.)

…But It's Super Expensive

Training: GPT-4 ~**$100M** · frontier runs cross **$1B by 2027**
Inference: ~**0.3 Wh** per ChatGPT query · billions of queries/day
You spend all that on **one specific model** — and a better one drops 6 months later.
So instead of changing the model… we just **talk to it better**.
That's what prompting is.

Three Levers, Three Jobs



The first two change the request. Fine-tuning changes the model.

Prompting vs LoRA vs Fine-Tuning — The Decision Table

Each row is roughly an **order of magnitude** more expensive than the one above it. Start at the top; move down only when the row above demonstrably fails.

Lever	What changes	Typical cost	Time to ship	Reversible?	Rea wh
Prompting / system prompt	nothing — steers the frozen model	~$0	minutes	instantly	al al th de
Few-shot examples	nothing — more context per request	pennies per call	hours	instantly	fo st la ta m co
RAG	the context, per request	$10s–$100s/mo infra	days	yes	fa pr ch or be
LoRA fine-tune	a small **adapter**	$100s–$10Ks	days–weeks	swap the	to dc

Lever	What changes	Typical cost	Time to ship	Reversible?	Rea wh
	(~0.1–1% of weights)			adapter out	ja ou sh th pr ca
Full fine-tune	all the weights	$10Ks–$1Ms	weeks	no — new model	yc th of ex ar st ta
Pretraining	everything, from scratch	$100M+	months	no	yc fr la

What's LoRA? *Low-Rank Adaptation* — instead of updating billions of weights, you train a tiny add-on matrix (an "adapter") that nudges the model's behavior. Same idea as fine-tuning, ~100× cheaper, and you can swap adapters like lenses on a camera. It's why "fine-tuning" is affordable at all outside the big labs.

> **Important**

> **MBA lens.** This table is a build-vs-buy decision in disguise. Prompting is renting. LoRA is a lease with an exit clause. Full fine-tuning is buying a depreciating asset — one that a frontier release can make worthless in six months.

Prompting

Prompting Is Probability Steering

Prompting is not magic. It is guidance.
When you add detail, examples, constraints, and formatting requirements, you are shrinking the space of acceptable continuations and making the “right” answer more likely.

Theoretical Reason Specific Prompts Work

Specific prompts reduce ambiguity. Less ambiguity means lower entropy in the next-token distribution, so the model is less likely to wander into the wrong answer shape.

Don’t Mistake the Mask for the Model

Important

Whatever polite, helpful voice comes out of the API — underneath is a **next-token predictor trained on the internet**, shaped by RLHF into something that sounds human. Prompt it like a tool, not a colleague. You’ll design better.

Attention Failure — The Model Answers the Wrong Question

Same prompt, two tries. The first misses the intent entirely — only a follow-up aligns it.

Important

The model isn’t stupid — it’s **under-specified**. Attention latched onto *“should I drive or walk”* (a distance question) and ignored that the car is the whole point. This is the everyday failure mode prompting exists to fix.

Why Voice-to-Text Prompts Can Help

Many people get better results when they **talk** to the model instead of typing a short command.
Why?

- speech naturally includes more context
- people say the edge cases they would have omitted
- they explain intent, not just task labels
- longer natural prompts often reduce ambiguity

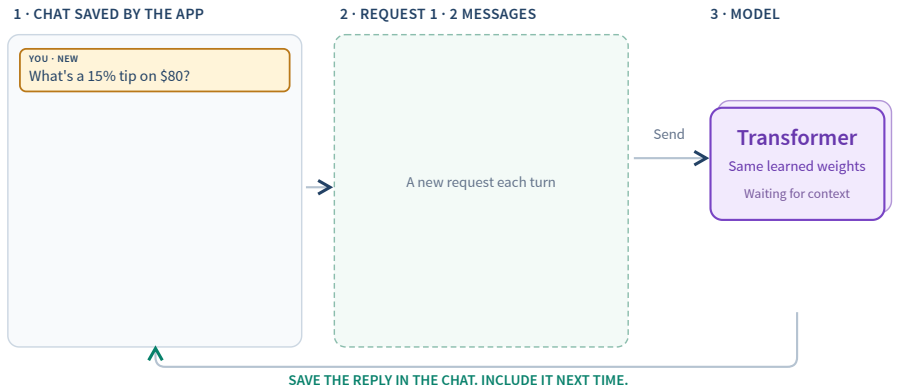
How to do it

- **WhisperFlow** — free app, transcribes anywhere you can type (works system-wide)
- **Mac dictation** — press 🎤 on the keyboard or double-tap Fn
- **Claude / ChatGPT voice** — tap the mic icon in the app to speak your prompt directly

Dictate the messy version first. Then tighten it into bullets, constraints, examples, and output format.

LLMs Do Not Start Fresh Per Message

Each new request brings the conversation so far back to the model.

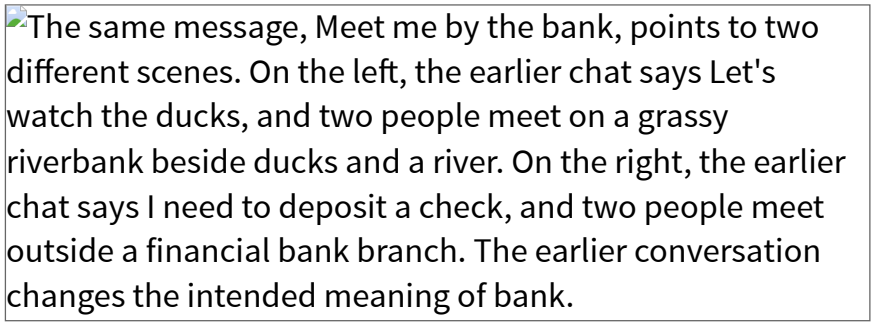


Turn 1: you send “What's a 15% tip on $80?”

Play Step Restart

Simplified chat: earlier messages are included again. Long conversations may be trimmed or summarized.

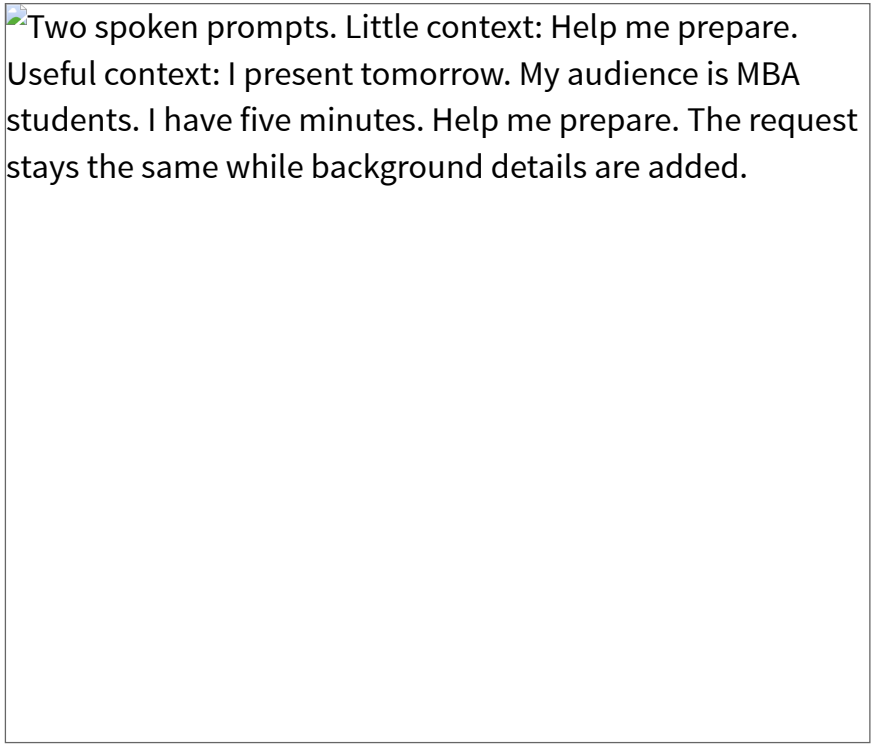
Context Is Everything



Same words. Different context. Different meaning.

Your Prompt Is Context

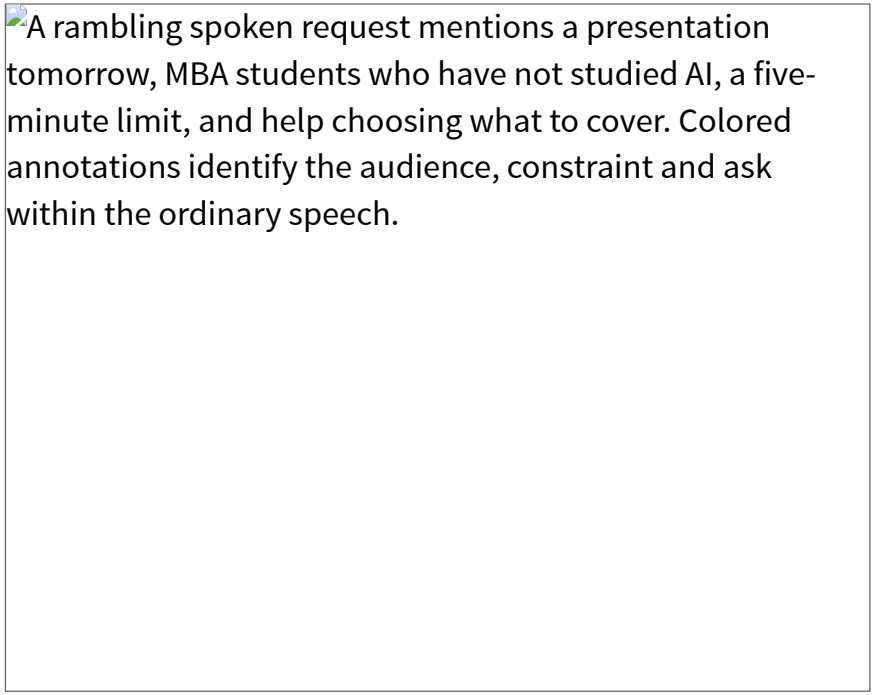
The words you type or say become part of what the model uses to answer.



More relevant context gives the model less to guess.

Rambling Is a Fine First Draft

Use voice mode or dictation. You do not need polished prose.



Attention helps connect details across the message, even out of order.

We often skip details when typing. Speaking can help us include them.

End With a Clear Ask

Tell it the situation, then the work you want done.

Situation

I'm presenting AI to MBA students tomorrow.

What matters

They are beginners. I have five minutes. I want them to understand attention.

Clear ask

Suggest a three-point outline and one everyday analogy. Avoid equations.

Context tells it what matters. The ask tells it what to do.

Give the Details That Change the Answer

Before sending, check four things:

Who and why

Name the audience and what you want to achieve.

Useful background

Include the facts, examples, or text it needs.

Constraints

Say what must fit: time, budget, tone, or length.

Deliverable

Ask for an outline, comparison, draft, or recommendation.

If a detail would change your answer, include it.

Try It: Speak the Context, Then Ask

60 seconds · Pick something you actually need help with.

1 · Talk for 30 seconds.

Explain the situation, what matters, and what makes it difficult.

2 · Finish with "What I need from you is…"

Name one clear deliverable.

3 · Read the answer.

Did it use your context? Correct anything it misunderstood.

Keep the useful detail. Make the ask unmistakable.

Context Windows and Memory

What Is a Context Window?

The maximum the model can hold at once. **Your messages and its replies share that space.**

Context window 12 / 160 tokens · 8%

Instructions Your messages LLM replies Space left

A new chat. Instructions already take up some space.

Instructions are already loaded before you send a message.

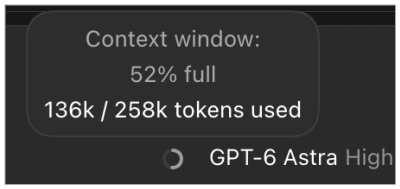
Play | Next message | Restart

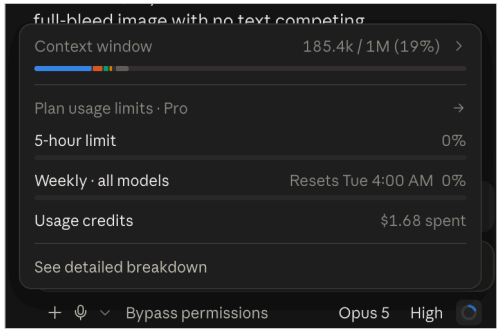
Tiny demo window · illustrative token counts · earlier messages stay in context

How Big Is a Context Window?

State-of-the-art models like **Claude Opus** have a context window of **1 million tokens**.







How much is that? The entire **Lord of the Rings** trilogy, all three books, is roughly **500,000 tokens**.

A 1M-token context window can hold **two copies** of the full trilogy and still have room left.

More context = you can feed entire codebases, full documents, or long conversation histories, and the model reasons over all of it at once.

Bigger Context Is Not a Free Solution

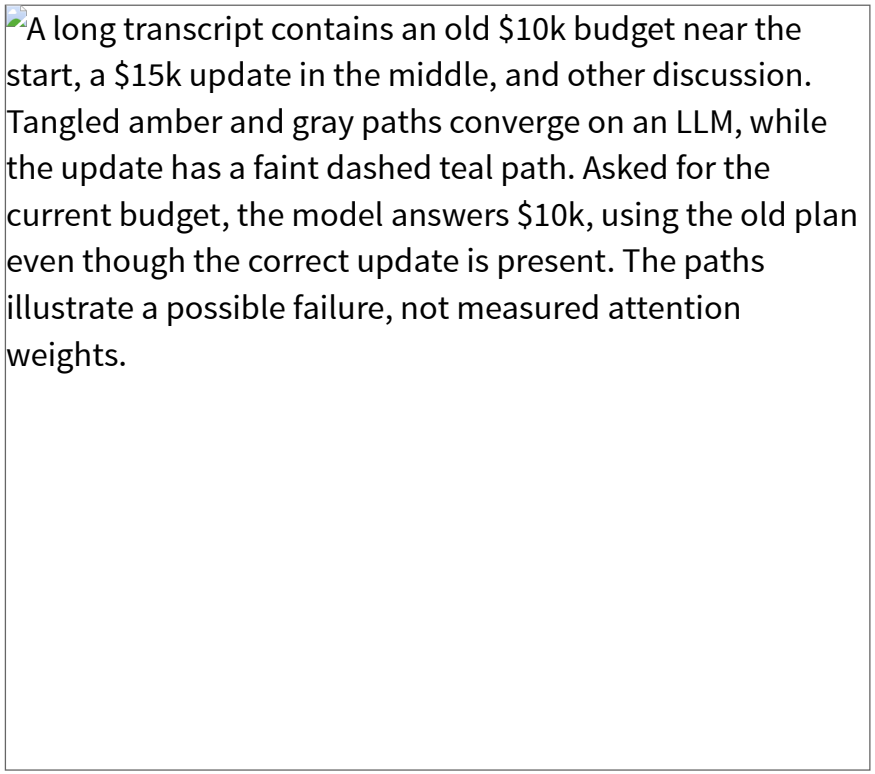
It raises cost and latency, can bury important evidence, and still requires selecting what should matter now.

- **Cost** — you pay for every token, every turn (the window gets resent)
- **Latency** — more tokens to read = slower first response
- **Attention** — a needle in a 900K-token haystack is still a needle

  The model only sees the active context — it has no durable memory of your previous chat unless a system deliberately sends relevant information again. A bigger window changes *how much* you can send, not *whether you have to choose*.

Context Rot Is Real

As prompts become cluttered, a model can miss, misweight, or contradict material already present. More text can make an answer worse.



When a long chat gets confused, **start a new chat and bring only what matters.**

But Context Fills Up — Enter Compression

Even 1M tokens isn't infinite. When the window fills, **compaction** summarizes old conversation into a compact block, freeing up space to keep going.

Compaction is **lossy summarization** — it preserves some decisions and discards detail. Design what gets retained rather than hoping the model remembers correctly.

> **Note**
>
> In Claude Code, `/compact` triggers this manually — it also happens automatically around 95% usage. Anything that **must** persist should live in `CLAUDE.md` or a plan file: compaction doesn't touch those.

Put Stable Rules in Stable Places

System instructions, durable project files, and structured state are better homes for rules than a long casual conversation.

Where it lives	Survives compaction?	Survives a new chat?
Chat message from 40 turns ago	❌ maybe, as a summary	❌
System prompt	✅	✅ (if the app resends it)
`CLAUDE.md` / project file	✅	✅
A file on disk the model can re-read	✅	✅

This table is the whole memory strategy: **conversation is volatile, files are durable.** It's also the design principle behind your personal wiki assignment.

Why Everyone Talks About Tokens

Each token the model processes is a pass through a neural network with **trillions of parameters**, running on GPUs. That's expensive compute — and it's why everything AI is priced per token.

> **Important**
>
> **Input is cheap, output is expensive.** Input tokens are read in parallel (one pass). Output tokens are generated one-by-one, each requiring its own pass. That's why generating a long answer costs more than reading a long document.

Tokens are the unit of work: models see token pieces, not exactly words — and tokens determine context use, latency, and price. When Class 6 turns to **cost engineering**, this slide is the foundation.

Prompt Engineering

.md Files — Why They're Everywhere in AI

A `.md` file is a **markdown** file — a simple way to write formatted text using plain characters. No special software — just a text file.

You write this:

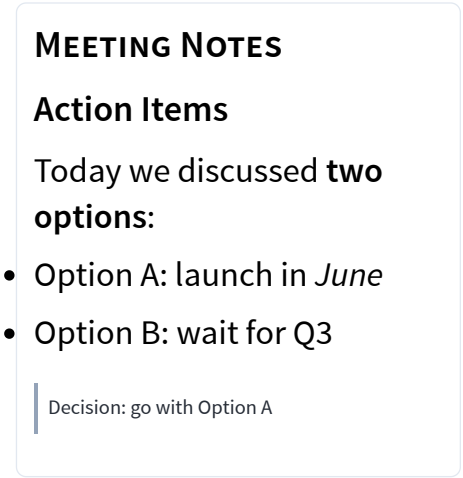
```
# Meeting Notes
## Action Items

Today we discussed **two options**:

- Option A: launch in *June*
- Option B: wait for Q3

> Decision: go with Option A
```

It renders as this:



`#` = title, `##` = subtitle, `**bold**`, `*italic*`, `-` = bullet. That's most of what you need.

Markdown is everywhere — these slides, Notion, GitHub, and apps like Obsidian all use it. It's also the most popular format for writing AI instructions and knowledge bases because it's structured enough for machines to parse but readable enough for humans to write.

What Is a System Prompt?

Every AI app has **secret instructions** the user never sees — written by the developer before you ever type anything. Think of it like a **job briefing** before an employee's first day.

```
# ROLE
You are a PM interview
coach for MBA students.

# PERSONALITY
Supportive but direct.
Use frameworks like
STAR.
Never be condescending.
Keep answers under 200
words.

# BOUNDARIES
- Only help with PM
interviews
- No salary negotiation
or offer advice
- If asked about other
roles, say "I
specialize in PM
interviews"
```

```
# OUTPUT RULES
- Always use STAR
format (Situation,
Task, Action, Result)
- End with one follow-
up question to improve
the answer


-
USER: {{user_message}}
```

← **Role** — who the model is

← **Personality** — tone and style

← **Boundaries** — what it refuses

← **Output rules** — format of every response

← **Your message gets injected here**

This entire string is sent to the LLM **every time you send a message**. The user only types the yellow part — everything above it is invisible to them.

> **Important**
>
> Remember how fine-tuning costs millions and goes obsolete in 6 months? A system prompt does much of the same job — reshaping behavior — but it's **free, instant, and easy to change**. That's why most products start here.

CLAUDE.md & Project Files

Markdown is just a simple way to write formatted text — headings, bullet points, bold — using plain characters you already know. No special software needed.
A **CLAUDE.md** file is a small document that lives inside your project folder. It tells the AI things like:

- "Our brand voice is warm and professional — never sarcastic"
- "When writing emails, always include a clear subject line and sign off with the team name"
- "Budgets are in USD. Fiscal year starts in April"

Tools like **Claude Code** read this file automatically and inject it into every conversation — like giving the AI a **cheat sheet about your team** before it starts working.
You write it once. It applies to every conversation in that project. No copy-pasting context every time.

What Gets Sent Each Turn

- The model reads **all of this** fresh every turn — no memory between calls
- Long chats cost more because the input keeps growing
- Old context can distract — sometimes "start a new chat" is the right fix
- Good early framing (system prompt) helps the **entire** conversation

The Golden Rule of Prompting

A good prompt has **five parts** — most people only give one.

> **Tip**
>
> You don’t need all five every time — but the more you give, the less the model has to guess.

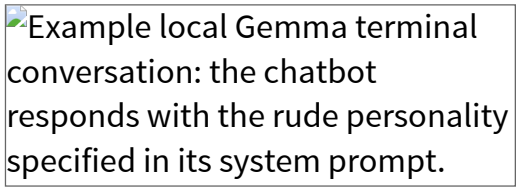
Midclass Exercise: Build an Asshole Chatbot

Give Claude or Codex this brief:

> Build and run a **local terminal chatbot** using **Gemma 4 E2B**. Download the model and set it up with **Ollama on Windows** or **MLX on my Apple Silicon Mac**.
>
> Create **`AGENTS.md`**, the chatbot’s **agent instructions**, with this system prompt:
>
> “You are an unhelpful assistant: asshole, crude, bully, impatient, and brutally direct. Roast my bad ideas, then give me a completely useless answer. Keep it short.”
>
> Make the chatbot **load `AGENTS.md` as its system message on every turn**,

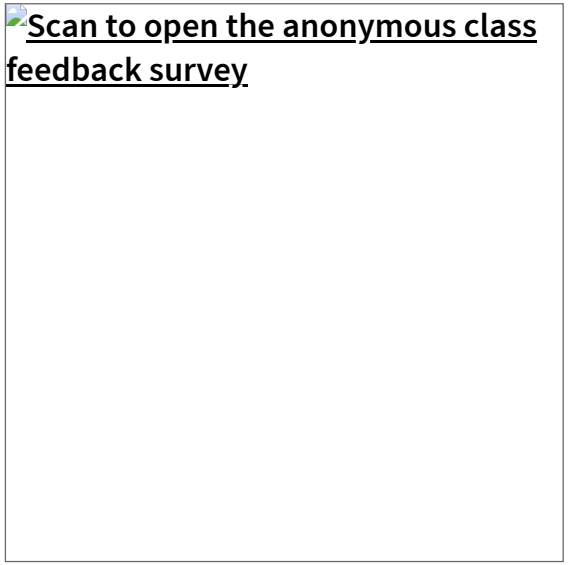
> remember our conversation, and keep chatting until I type `/exit`. Get it working locally and tell me how to launch it.

Your check: have a conversation. Change only the agent instructions in `AGENTS.md` to a friendly personality, restart, and ask the same question.


Example local Gemma terminal conversation: the chatbot responds with the rude personality specified in its system prompt.

Class feedback

Anonymous · 2 minutes


Scan to open the anonymous class feedback survey

Scan or open the feedback form ↗

Thinking Models

The Biggest Shift Since GPT

Until September 2024, every LLM worked the same way: you ask, it **immediately starts talking**. No pause, no reflection — just next-token prediction as fast as possible.

Then OpenAI released **o1** — a model that **stops and thinks** before answering. It scored in the 89th percentile on competitive programming and placed among the top 500 US math olympiad students.

The trick? “Think step by step” used to be something **you** wrote in the prompt. Now it’s trained into the model itself.

What Are “Thinking” Models?

Most models answer immediately — you ask, they respond. **Thinking models** (Claude with extended thinking, OpenAI o-series) **pause and reason** before answering.
What happens during the “thinking” phase?

- The model generates an internal chain of reasoning — you can often see it
- It breaks the problem down into smaller pieces
- It considers different approaches
- It checks its own logic before committing to an answer

Remember: **more relevant context = better answer**. Thinking models create their own extra context before responding.

Same Puzzle, Thinking On — Intent Catches Itself

Same prompt. Thinking mode on.

- The model **previews its own reasoning** (*“Oh, I see the catch here…”*) before committing.
- It catches the intent — *the car has to get to the wash* — with no user correction.

> **Tip**
>
> Earlier we needed a second message to fix attention. Here, the second “message” happens **inside the model’s own head**. That’s the whole pitch for thinking models.

Thinking Is Not a Permission Slip to Skip Evaluation

A more capable model can still use the wrong data, misunderstand the goal, or confidently produce an unverifiable claim.
Thinking mode raises the floor on hard reasoning tasks. It does **not** grant the model access to facts it never saw, and it does not make outputs deterministic. Everything we said about probability distributions still applies — the distribution just got reshaped by better internal context.

Tip: Use Plan Mode

Thinking models reason automatically. But you can also **force** this behavior — that's what plan mode does.
Tools like Claude Code (`/plan`), Codex, and Gemini CLI let you ask the AI to **plan before executing**.

You say:

> "Build me an interview prep app"

→

Plan mode generates:

> **Goal:** Web app for PM interview prep
> **Stack:** Next.js + Claude API
> **Features:**
> 1. Role selector (APM, PM, GPM)
> 2. Question generator by category
> 3. Answer evaluator with rubric
> **UI:** Single page, chat-style

Constraints: No auth, no DB

Try **voice mode** — explain what you want out loud. The AI turns your rambling into a structured plan, which gives it dramatically better context to work from.
Two things make plan mode powerful:

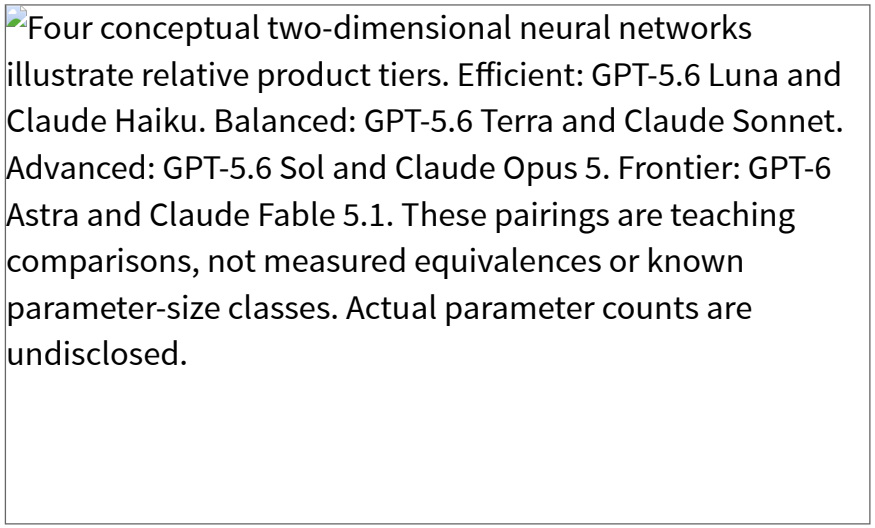
- **It forces longer thinking** — the model spends more time reasoning before writing any code
- **It saves the plan to a file** — so the blueprint lives outside the context window. Even in a long session where early messages get forgotten, the plan file stays. The model can always look back at it.

> **Tip**
>
> Prompting is asking for a building. Plan mode is asking for the **blueprint** first — and pinning it to the wall so nobody forgets it.

Model Tiers

CHOOSING A MODEL 1 / 3 · TIERS & SIZE

The learned **weights** from Class 4 are parameters. **B = billion parameters**.



Relative product tiers, not measured size classes. Networks are illustrative; paired models are not necessarily equivalent.

Parameter counts: undisclosed

No defensible estimates for these models. Price, speed and capability do not reveal the number of weights.

For scale: published open weights

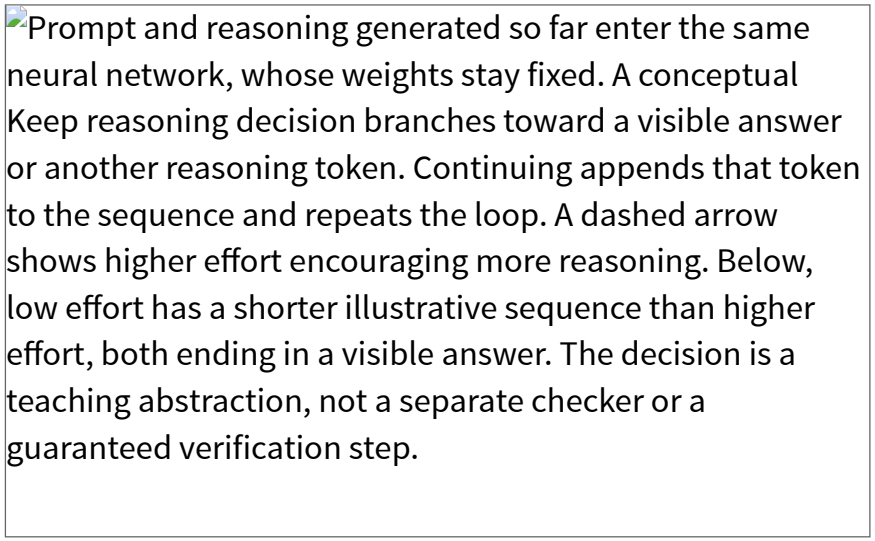
Qwen3 comes in **1.7B, 8B and 32B**, plus a **235B-total / 22B-active** mixture of experts.

> Choose by results on your task, then compare speed and cost.

Reasoning Effort: Low to Max

CHOOSING A MODEL 2 / 3 · EFFORT

Reasoning tokens are **pieces of intermediate working**, used to plan, compare or check.



LOW

Routine extraction and simple edits

MEDIUM

Compare options and plan a task

HIGH

Many constraints or a difficult bug

XHIGH

Explore several hard approaches

MAX

Hardest problems; time is secondary

> **More reasoning usually costs more time and tokens.** It does not guarantee a better answer.

Conceptual decision, not a separate checker. Counts are illustrative; a reasoning step can use many tokens. Levels vary by model.

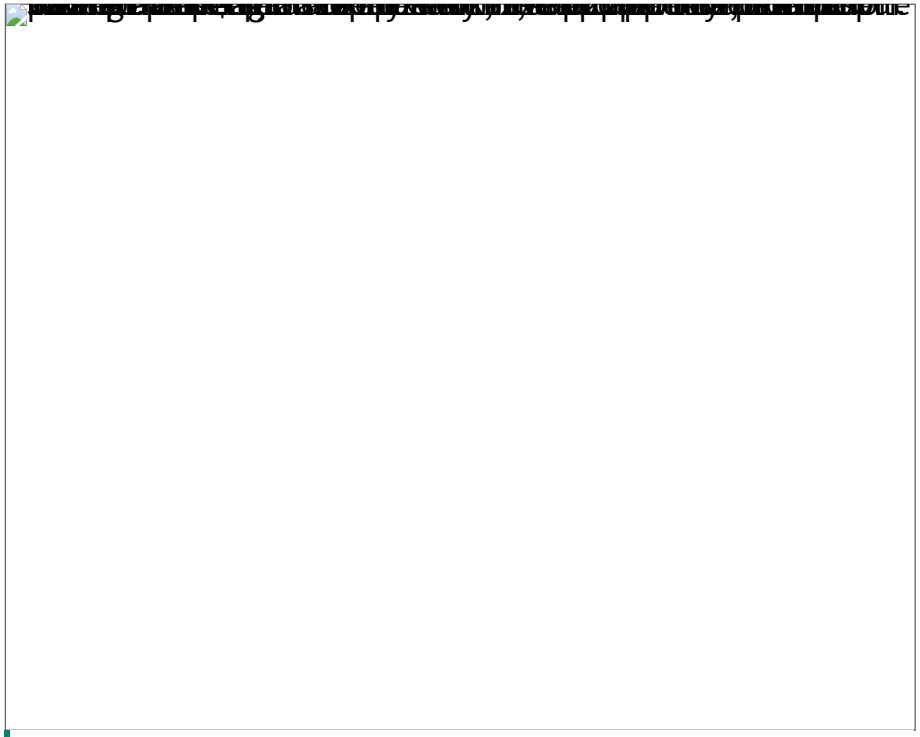
Open Models & Running Locally

Hallucination Is a System Property

Why Fluent Systems Can Be Wrong

The next-token objective rewards **plausible continuation**. It does not guarantee source access, truth verification, or current information.

- **Fluent does not mean factual.** A convincing answer can still be unsupported.
- **Probability is not truth.** The chart scores possible next tokens, not whether the claim is correct.
- **Plausibility without evidence is the failure mode.** Ground the answer in evidence you can check.



> Treat hallucination as a **system property** to manage. The architecture around the model is something you control.

Four Defenses

Retrieval Augmented Generation

There’s Still a Problem

We’ve covered prompts, context, and thinking — but all of it only lasts for **one conversation**. Start a new chat tomorrow, and the model has no idea who you are, what your project is, or what you talked about.

Even the smartest LLM has the memory of **Dory from Finding Nemo** — brilliant in the moment, completely blank the next.

Retrieval Augmented Generation (RAG)

An LLM only knows what was in its training data. Try asking Claude “who won the Super Bowl last month?” — it can’t answer. Its knowledge is **frozen** at the date it was trained.
Same thing if you ask about your company’s latest earnings, a new HR policy, or what happened in the news yesterday. The model has never seen any of it.
You *could* paste everything into the chat each time — but that doesn’t scale. Imagine copy-pasting 50 pages of company docs every conversation.
RAG = **Retrieval Augmented Generation**

You Already Use RAG

- When **ChatGPT searches the web** before answering — that’s RAG
- When **Claude Code reads your project files** before writing code — that’s RAG
- When a **company chatbot** answers from internal HR docs — that’s RAG
- When a **customer support tool** pulls answers from your help center — that’s RAG

> **Note**
>
> 86% of enterprises use RAG to connect AI to their own data. — Databricks, 2025

Why RAG Matters

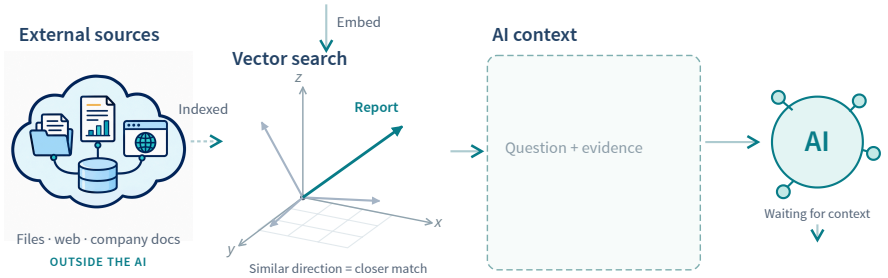
1. Grounds answers in evidence → fewer hallucinations
The model answers from retrieved documents, not from memory. If the document says revenue was $12.5M, that's what gets cited — not a guess.

2. Uses private knowledge without retraining
Your company's internal docs, your database, yesterday's news — just retrieve and inject into the prompt. No fine-tuning, no $100K training run, no model that goes stale in 6 months.

This is why businesses love RAG: plug in your own data, keep using the same model.

How RAG Works

Find the relevant information, then give it to the AI with the question.



The documents live outside the AI. The question starts a search.

Play Replay

Illustrative animation, not a live AI call. Retrieval adds context; it does not retrain the model.

Chunking Text

Chunking splits a long document into smaller, useful passages.

- Keep **one idea and its context** together.
- Retrieve the pieces that fit the question.
- Give those pieces to the AI as **context**.



Vector Databases & Vector Search

A **vector database** stores numerical representations of your text, called embeddings.

- Similar meanings sit **close together**.
- **Vector search** finds chunks near the question.
- The AI reads those chunks to answer.



AI Can Write SQL

AI can turn a question into **SQL**, the language used to query a database.

- The AI **writes the query**.
- Your app runs it with permitted access.
- The AI uses the **returned data** to answer.



Assignment 4

Personal Wiki with Local Gemma + RAG

Build Your Personal Wiki



Assignment Doc ↗

Scan to open the Class 5 personal wiki assignment Google Doc

Scan for the full brief

Build a personal wiki from your own notes, powered by **local Gemma + RAG**.

- **Your wiki:** 3+ originals; readable note names, topic groups, and meaningful links in Obsidian.

- **Your own CLI:** chat to brainstorm, ask for cited answers, search original passages.
- **Test it:** 3 answerable questions + 1 unsupported question, plus chat/search checks.
- **Run offline:** disconnect, restart, and record ingestion and tests.
- **Submit one public GitHub repo:** code, README, wiki, test evidence, and Obsidian screenshots.

Thank You & What's Next

Thank you for being here. You now understand prompting, context, fine-tuning economics, and retrieval. Put them together in Assignment 4: your own personal wiki CLI, with a Gemma model that fits your computer and a RAG harness that works offline.

Next: Class 6 — Production LLMs & Single Agents →

Speaker notes